

WEEK 7 Skill OSSP

PART 1 — Process Synchronization and Child Process Monitoring using waitpid()

1. Create a working folder

```
harshitha18@LAPTOP-6E22AR6C ~$ mkdir week7_s
harshitha18@LAPTOP-6E22AR6C ~$ cd ~/week7_s
harshitha18@LAPTOP-6E22AR6C ~/week7_s$ pwd
/home/harshitha18/week7_s
```

2. Create the C program and Compile the program

```
harshitha18@LAPTOP-6E22AR6C ~/week7_s$ nano waitpid_demo.c
harshitha18@LAPTOP-6E22AR6C ~/week7_s$ gcc waitpid_demo.c -o waitpid_demo
harshitha18@LAPTOP-6E22AR6C ~/week7_s$ ls -l waitpid_demo
-rwxr-xr-x 1 harshitha18 harshitha18 16312 Sep 17 18:17 waitpid_demo
```

3. Execute the program

```
harshitha18@LAPTOP-6E22AR6C ~/week7_s$ ./waitpid_demo
Parent: PID = 35
Child: PID = 36
Parent: Waiting for child PID = 36
Child: Performing some work...
Child: Work completed. Exiting...
Parent: Child PID 36 has terminated.
Parent: Child exited normally with status 5
```

4. Demonstrate waitpid() behavior

```
harshitha18@LAPTOP-6E22AR6C ~/week7_s$ ./waitpid_demo
Parent: PID = 35
Child: PID = 36
Parent: Waiting for child PID = 36
Child: Performing some work...
Child: Work completed. Exiting...
Parent: Child PID 36 has terminated.
Parent: Child exited normally with status 5
```

Summary:

The program successfully created a child process using `fork()`. The parent process used `waitpid()` to wait for the child to complete and then retrieved the child's exit status, demonstrating process synchronization and child process monitoring.

PART 2 — PATH Variable & Command Resolution

1. Checking your PATH

```
harshiitha18@LAPTOP-6E22AR6C~/.week7_s$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/mnt/c/Program Files (x86)/Common-Files/Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0/:/mnt/c/Program Files/NVIDIA Corporation/NVIDIA App/NvDLISR:/mnt/c/Program Files/Python311:/mnt/c/Users/VHV/AppData/Local/Programs/Microsoft Software/Python/Python311:/mnt/c/Users/VHV/AppData/Local/Programs/Git/cmd:/mnt/c/Users/VHV/AppData/Local/Programs/Microsoft Windows Common-Files/Universal C Runtime/bin
harshiitha18@LAPTOP-6E22AR6C~/.week7_s$ which ls
/usr/bin/ls
```

2. Create the PATH resolver program

```
harshitha18@LAPTOP-6E22AR6C ~/week7 $ nano path_resolver.c
harshitha18@LAPTOP-6E22AR6C ~/week7 $ gcc path_resolver.c -o path_resolver
harshitha18@LAPTOP-6E22AR6C ~/week7 $ ls -l path_resolver
-rwxr-xr-x 1 harshitha18 harshitha18 16344 Sep 17 19:00 path_resolver
```

3. Test command resolution

```
harshitha18@LAPTOP-6E22AR6C ~/week7_s$ ./path_resolver ls
Command to resolve: ls
Searching PATH directories...

Executable found: /usr/bin/ls
Permission: Executable
```

4. Test a missing command

```
harshiitha18@LAP-TOP-6E22AR6C ~/week7_s$ ./path_resolver abcxyz
Command to resolve: abcxyz
Searching PATH directories...

Command not found in PATH.
```

5. Test another real command

```
harshitha18@LAPTOP-6E22AR6C ~/week7_s$ ./path_resolver gcc
Command to resolve: gcc
Searching PATH directories...

Executable found: /usr/bin/gcc
Permission: Executable
```

Summary:

The program successfully retrieved the PATH variable and searched its directories to locate executable commands. It verified executable permissions and correctly reported when a requested command was not found.